



PROGRAMACIÓN ORIENTADA A OBJETOS

Getters y Setters en Java

Encapsulamiento y control de acceso seguro en POO

Ma.Alejandra Ramirez y Kevin Olave

Índice de Contenido

Estructura

01

¿Qué son?

Concepto básico y encapsulamiento

02

Acceso Directo vs Controlado

Validación y seguridad de datos

03

Sintaxis Getter / Setter

Estructura de lectura y escritura

04

Diagrama UML

Estructura de clases y visibilidad

05

Ejemplo Práctico

Implementación en Java

06

Conclusión

Buenas prácticas en POO

¿Qué son los getters y setters?

🕒 **Getter (Lectura)**

Método público que devuelve el valor de un atributo privado. Ej: `getNombre()`.

✎ **Setter (Modificación)**

Método público que asigna o actualiza el valor de un atributo de forma controlada. Ej: `setEdad(int edad)`.

El Pilar del Encapsulamiento

Permiten leer y modificar atributos privados de una clase sin exponerlos directamente al exterior.

- ✓ Protección de datos internos
- ✓ Validación previa en asignaciones
- ✓ Estándar universal en POO

Acceso Directo vs. Acceso Controlado

⚠ Sin Encapsulamiento

```
public class Estudiante {  
    public int edad;  
}  
  
estudiante.edad = -5;
```

Cualquier parte del programa puede asignar un valor inválido directamente, sin validación.

🛡 Con Encapsulamiento

```
private int edad;  
  
public void setEdad(int edad) {  
    if (edad > 0) {  
        this.edad = edad;  
    }  
}
```

El setter valida antes de asignar. Un valor inválido se rechaza automáticamente.

Diagrama de Clases: Estudiante

Estudiante

- nombre: String
- edad: int
- promedio: double

- + Estudiante(nombre, edad, promedio)
- + getNombre(): String
- + getEdad(): int
- + getPromedio(): double
- + setNombre(nombre): void
- + setEdad(edad): void
- + setPromedio(promedio): void

- Atributos Privados

Ocultos al exterior para evitar modificaciones directas o corrupción de datos.

+ Métodos Públicos

Constructores, getters y setters que sirven como interfaz controlada de acceso.

Uso en Código Java

```
Estudiante e = new Estudiante(
    "Laura Gómez", 20, 4.5);

System.out.println(
    e.getNombre());

e.setPromedio(4.8);

e.setEdad(-5);
// rechazado por el setter
```

1. Lectura segura

`getNombre()` obtiene el valor sin exponer el campo.

2. Escritura controlada

`setPromedio()` actualiza cumpliendo reglas.

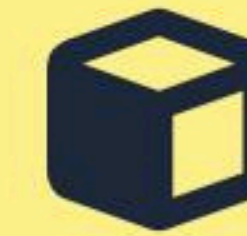
3. Rechazo de anomalías

`setEdad(-5)` demuestra el bloqueo de datos inválidos.

Conclusión

Beneficios Clave

- ✓ Mantiene atributos privados controlando lectura y modificación.
- ✓ Evita ingreso de datos inválidos mediante validaciones en setters.
- ✓ Oculta la implementación interna sin romper el resto del código.



Práctica Estándar

Son la base absoluta del encapsulamiento en lenguajes orientados a objetos como Java.



¡Muchas Gracias!

Programación Orientada a Objetos